

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №1
з курсу «Машинне навчання»
на тему:
«Лінійна регресія, оптимізація та статистичний аналіз»

Виконав:
студент групи ФЕІ-42
Трофимов Володимир
Перевірив:
доц. Сінькевич О. О.

Мета

Зрозуміти архітектуру лінійної регресії, навчитися самостійно реалізовувати алгоритми оптимізації, обирати інформативні ознаки, обчислювати метрики якості та перевіряти статистичні припущення моделі.

Вхідні дані

- Ames Housing Dataset:
 - цільова ознака SalePrice;
 - числові ознаки: LotArea, GrLivArea, YearBuilt тощо;
 - Категоріальні ознаки: Neighborhood, BldgType, HouseStyle, KitchenQual тощо.

Завдання

Створити ірyпb файл для виконання таких цілей:

- Дослідження даних, відбір ознак та попередня обробка:
 - Первинний аналіз даних;
 - Відбір ознак;
 - Розбиття вибірки;
 - Кодування категорій;
 - Масштабування.
- Реалізація лінійної регресії засобами numpy;
- Градієнтний спуск: Batch GD, SGD, Mini-batch GD;
- Оцінка якості та коефіцієнт детермінізації;
- Перевірка статистичних припущень.

Теоретичні відомості

1. Лінійна регресія

Лінійна регресія є методом машинного навчання, який використовується для прогнозування числової цільової змінної на основі одного або декількох вхідних параметрів. У цій лабораторній роботі як цільову змінну використано SalePrice — ціну продажу будинку з набору даних Ames Housing.

У загальному випадку лінійна модель задається виразом:

$$\hat{y} = Xw + b$$

де (X) — матриця вхідних ознак, (w) — вектор вагових коефіцієнтів моделі, (b) — вільний член, а ($\hat{y}$) — прогнозоване значення цільової змінної.

Модель намагається підібрати такі значення (w) та (b), за яких прогноз максимально наближений до фактичних значень (y).

Для оцінювання помилки прогнозування використовується середньоквадратична помилка (Mean Squared Error, MSE):

$$MSE = (1/n) \times \sum (y_i - \hat{y}_i)^2$$

де (n) — кількість об'єктів, (y_i) — фактичне значення, а $(\hat{y}_i)$ — прогноз моделі.

Чим менше значення MSE, тим менша середня квадратична помилка прогнозування.

2. Градієнтний спуск

Для пошуку оптимальних параметрів лінійної моделі можна використовувати метод градієнтного спуску (Gradient Descent). Його основна ідея полягає у поступовій зміні параметрів моделі в напрямку, протилежному градієнту функції втрат.

Загальне правило оновлення параметрів має вигляд:

$$\theta = \theta - \alpha \times \nabla J(\theta)$$

де (θ) — параметри моделі, (α) — швидкість навчання (learning rate), $(J(\theta))$ — функція втрат, а $(\nabla J(\theta))$ — її градієнт.

Для лінійної регресії з MSE градієнти параметрів визначаються як:

$$\partial MSE / \partial w = -(2/n) \times X^T (y - \hat{y})$$

$$\partial MSE / \partial b = -(2/n) \times \sum (y_i - \hat{y}_i)$$

Вибір швидкості навчання має суттєве значення. Якщо вона занадто мала, оптимізація відбувається повільно. Якщо значення надто велике, алгоритм може почати коливатися навколо мінімуму або взагалі розійтися. У лабораторній роботі значення learning rate підбирається за допомогою валідаційної вибірки, не використовуючи тестові дані.

3. Batch, Stochastic та Mini-batch Gradient Descent

Існує декілька варіантів градієнтного спуску, які відрізняються кількістю навчальних об'єктів, що використовуються для одного оновлення параметрів.

Batch Gradient Descent використовує всю навчальну вибірку для розрахунку одного градієнта. Такий підхід забезпечує найбільш стабільний напрям оптимізації, але кожне оновлення потребує обробки всіх навчальних об'єктів.

Stochastic Gradient Descent (SGD) використовує лише один навчальний приклад для кожного оновлення параметрів. Це дозволяє виконувати оновлення дуже часто та швидко рухатися на початкових етапах оптимізації, однак траєкторія навчання стає значно більш шумною.

Mini-batch Gradient Descent використовує невелику групу навчальних прикладів — mini-batch. Цей підхід поєднує переваги двох попередніх методів: він має менше коливань, ніж SGD, але виконує оновлення частіше, ніж повний Batch Gradient Descent.

Таким чином, Batch GD характеризується найбільшою стабільністю, SGD — найбільшою варіативністю оновлень, а Mini-batch GD є практичним компромісом між швидкістю та стабільністю оптимізації. У виконаному експерименті саме Mini-batch GD продемонстрував найкраще поєднання цих характеристик.

4. Поділ даних на вибірки

Для коректної оцінки моделі набір даних розділяється на навчальну, валідаційну та тестову вибірки.

Training set використовується для навчання моделі та оцінювання параметрів попередньої обробки.

Validation set використовується для порівняння моделей та вибору таких параметрів, як швидкість навчання.

Test set використовується лише для остаточного оцінювання якості вже вибраної моделі.

Важливим принципом є те, що тестова вибірка не повинна впливати на процес навчання або вибір параметрів моделі.

5. Data Leakage

Data leakage, або витік даних, виникає тоді, коли інформація з валідаційної чи тестової вибірки прямо або опосередковано потрапляє в процес навчання моделі.

Наприклад, якщо масштабування або заповнення пропущених значень виконується на всьому наборі даних до його розділення на Train і Test, статистичні характеристики тестових об'єктів впливають на параметри попередньої обробки. У результаті оцінка якості моделі стає некоректною.

Тому в лабораторній роботі спочатку виконується поділ даних, після чого операції попередньої обробки навчаються тільки на Training set. Для Validation та Test використовується вже отримане перетворення. Такий підхід дозволяє уникнути витoku інформації.

6. Обробка категоріальних ознак

У наборі Ames Housing присутні як числові, так і категоріальні ознаки. Категоріальні змінні не можна безпосередньо подавати до лінійної моделі у вигляді текстових значень.

Для їх перетворення використовується One-Hot Encoding. Цей метод створює окрему бінарну ознаку для кожної категорії. Наприклад, якщо ознака має категорії А, В та С, вони можуть бути представлені трьома бінарними змінними.

Перевагою One-Hot Encoding є те, що він не створює штучного порядку між номінальними категоріями. Наприклад, категорія В не вважається математично більшою за категорію А.

Якщо у Validation або Test зустрічається категорія, якої не було у Training set, кодер повинен коректно обробляти таку ситуацію. У роботі для цього використовується параметр `handle_unknown="ignore"`.

7. Масштабування ознак

Масштабування використовується для приведення числових ознак до порівнянного масштабу. Одним із поширених методів є стандартизація:

$$x' = \frac{x - \mu}{\sigma}$$

де (x) — початкове значення ознаки, (μ) — її середнє значення, а (σ) — стандартне відхилення.

Масштабування має особливе значення для градієнтної оптимізації. Якщо різні ознаки мають суттєво різні масштаби, поверхня функції втрат може мати витягнуту геометрію. У такому випадку градієнтний спуск може рухатися зигзагоподібно та повільно наближатися до оптимуму.

Після стандартизації поверхня оптимізації стає краще обумовленою, що дозволяє алгоритму рухатися до мінімуму ефективніше. У лабораторній роботі експеримент із масштабуванням підтверджує покращення оптимізації: валідаційна вартість для стандартизованих ознак становила 0.0591, тоді як для налаштованих немасштабованих ознак — 0.1766.

8. Коефіцієнт детермінації (R^2)

Для оцінювання якості прогнозування використовується коефіцієнт детермінації (R^2):

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

де (y_i) — фактичне значення, $(\hat{y}_i)$ — прогнозоване значення, а $(\bar{y})$ — середнє значення цільової змінної.

(R^2) показує, наскільки добре модель пояснює варіацію цільової змінної порівняно з базовою моделлю, яка для всіх об'єктів прогнозує середнє значення.

Чим ближче (R^2) до 1, тим кращою є відповідність моделі даним. Значення $(R^2 = 0)$ означає, що модель не краща за прогнозування середнім значенням. Значення менше нуля означає, що модель прогнозує гірше за такий базовий підхід.

Різниця між Train, Validation та Test (R^2) також може використовуватися для аналізу перенавчання. Значне падіння якості на тестових даних порівняно з навчальними може свідчити про overfitting або відмінність розподілів даних.

9. Залишки моделі

Залишком (residual) називається різниця між фактичним та прогнозованим значенням:

$$e_i = y_i - \hat{y}_i.$$

Аналіз залишків дозволяє перевірити, наскільки адекватно лінійна модель описує досліджувані дані.

Для цього використовуються графіки залишків від прогнозованих значень, гістограма залишків та Q-Q plot.

Якщо на графіку залишків спостерігається виражена криволінійна структура, це може свідчити про наявність нелінійної залежності, яку лінійна модель не враховує.

Якщо розкид залишків збільшується зі зростанням прогнозованого значення, це може свідчити про гетероскедастичність, тобто неоднакову дисперсію помилок.

Окремі дуже віддалені точки можуть вказувати на наявність впливових або аномальних спостережень. У виконаному аналізі було виявлено збільшення розкиду залишків для дорогих будинків та наявність викидів.

10. Перевірка статистичних припущень

Для додаткової оцінки розподілу залишків може використовуватися тест Шапіро–Вілکا. Він перевіряє відповідність даних припущенню про нормальний розподіл.

Водночас статистичний тест не повинен розглядатися ізольовано. На великих вибірках він може бути надзвичайно чутливим навіть до невеликих відхилень від нормальності, тому його результати доцільно доповнювати візуальним аналізом гістограми та Q-Q plot.

Отже, комплексний аналіз лінійної регресії включає не лише отримання високого значення (R^2), а й перевірку поведінки залишків, стабільності оптимізації, коректності попередньої обробки та відсутності витоку даних.

Хід роботи

Етап 1: Дослідження даних, відбір ознак та попередня обробка.

Виконаємо первинний аналіз даних.

```

Dataset path: data\train.csv
Shape: (1460, 81)
  Id  MSSubClass MSZoning LotFrontage LotArea Street Alley LotShape \
0  1         60      RL         65.0    8450   Pave   NaN     Reg
1  2         20      RL         80.0    9600   Pave   NaN     Reg
2  3         60      RL         68.0   11250   Pave   NaN    IR1

  LandContour Utilities  ... PoolArea PoolQC Fence MiscFeature MiscVal MoSold \
0         Lvl   AllPub  ...      0     NaN   NaN         NaN      0      2
1         Lvl   AllPub  ...      0     NaN   NaN         NaN      0      5
2         Lvl   AllPub  ...      0     NaN   NaN         NaN      0      9

  YrSold SaleType SaleCondition SalePrice
0   2008         WD         Normal    208500
1   2007         WD         Normal    181500
2   2008         WD         Normal    223500

[3 rows x 81 columns]

Dtype counts:
str         43
int64       35
float64      3
Name: count, dtype: int64
Numerical columns: 38
Categorical columns: 43

```

Рис.1 – Первинний аналіз даних

Як видно з рисунку 1, початковий набір містить 1460 об'єктів та 81 ознаку. Серед них 38 числових та 43 категоріальні ознаки. Також було досліджено ознаки, де були пропущені записи. Рейтинг таких ознак:

- PoolQC — 99.52%;
- MiscFeature — 96.30%;
- Alley — 93.77%;
- Fence — 80.75%;
- MasVnrType — 59.73%;
- FireplaceQu — 47.26%.

На цьому етапі вибрано декілька важливих числових ознак для статистичного аналізу.

Columns with missing values:		
	count	percent
PoolQC	1453	99.520548
MiscFeature	1406	96.301370
Alley	1369	93.767123
Fence	1179	80.753425
MasVnrType	872	59.726027
FireplaceQu	690	47.260274
LotFrontage	259	17.739726
GarageType	81	5.547945
GarageYrBlt	81	5.547945
GarageFinish	81	5.547945
GarageQual	81	5.547945
GarageCond	81	5.547945
BsmtExposure	38	2.602740
BsmtFinType2	38	2.602740
BsmtQual	37	2.534247

Descriptive statistics for selected variables:						
	SalePrice	GrLivArea	OverallQual	YearBuilt	LotArea	TotalBsmtSF
count	1460.0	1460.00	1460.00	1460.00	1460.00	1460.00
mean	180921.2	1515.46	6.10	1971.27	10516.83	1057.43
std	79442.5	525.48	1.38	30.20	9981.26	438.71
min	34900.0	334.00	1.00	1872.00	1300.00	0.00
25%	129975.0	1129.50	5.00	1954.00	7553.50	795.75
50%	163000.0	1464.00	6.00	1973.00	9478.50	991.50
75%	214000.0	1776.75	7.00	2000.00	11601.50	1298.25
max	755000.0	5642.00	10.00	2010.00	215245.00	6110.00

Рис.2 – Статистичний аналіз певних числових ознак

Додатково побудовано графіки розподілу даних та залежності основних характеристик від SalePrice. Зокрема, досліджувались залежності між ціною та GrLivArea, OverallQual та YearBuilt.

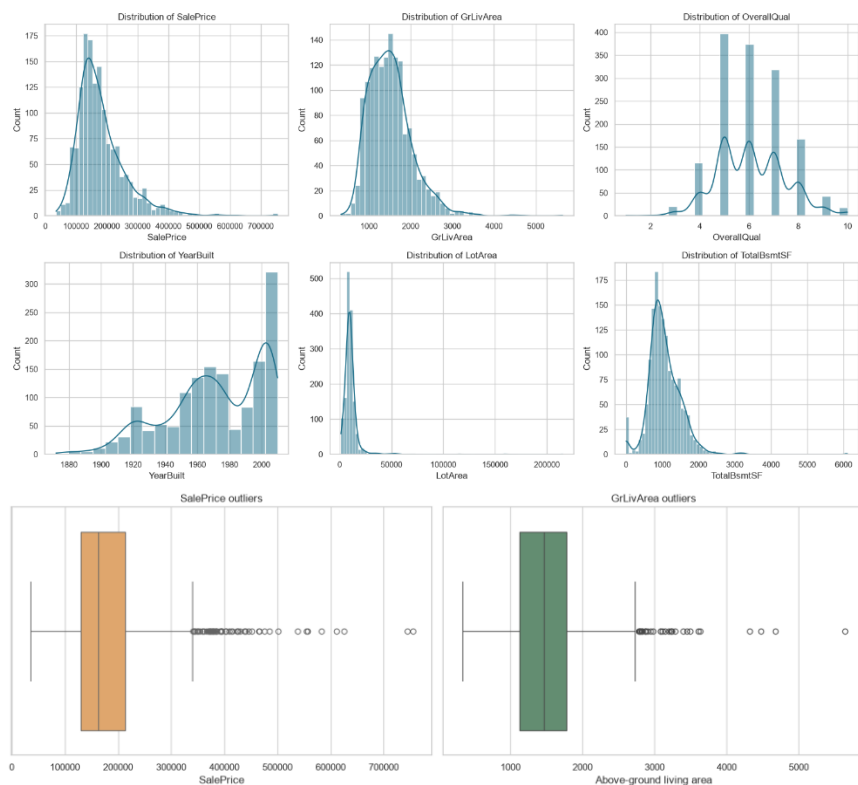


Рис.3 – Графіки розподілу числових даних

Аналіз показав, що SalePrice має виражену правосторонню асиметрію (рис.3), а між ціною та основними характеристиками будинку спостерігається залежність (рис.4).

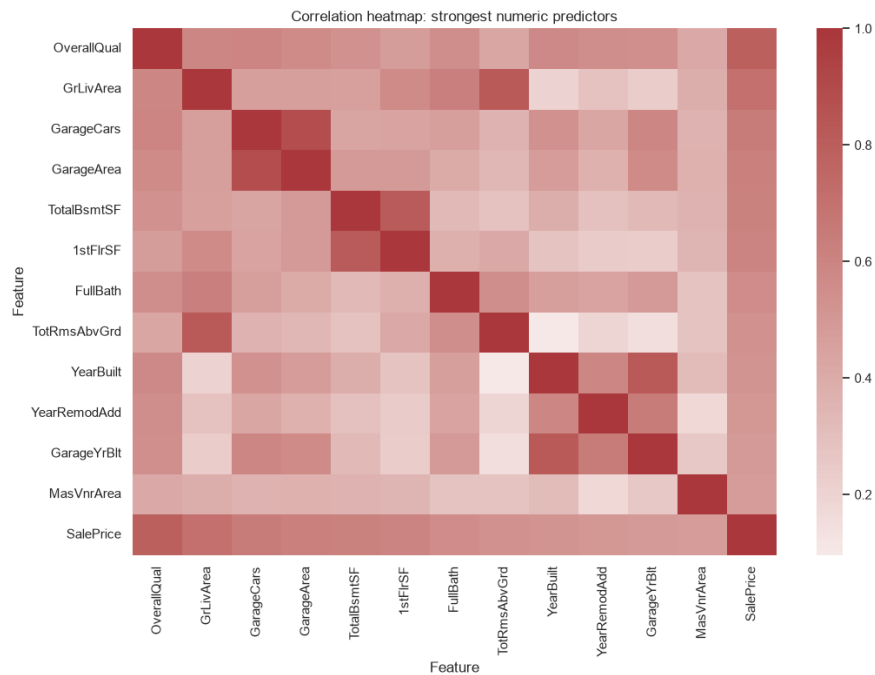


Рис.4 – Графік кореляції ознак до цільової ознаки

Після первинного аналізу було виконано відбір ознак, які будуть використовуватися для побудови моделі. Важливою вимогою було не використовувати інформацію з тестової вибірки під час вибору ознак. Тому спочатку початкові дані було розділено на навчальну та відкладену частини, після чого кореляції для вибору числових ознак розраховувалися тільки на навчальній вибірці.

```
Train/validation/test sizes: 1022/219/219

Training-only correlations used for selection:
OverallQual    0.785
GrLivArea      0.689
GarageCars     0.643
GarageArea     0.622
TotalBsmstSF   0.590
1stFlrSF       0.583
FullBath       0.549
TotRmsAbvGrd   0.520
YearBuilt      0.512
YearRemodAdd   0.512
GarageYrBlt    0.474
Fireplaces     0.461
Name: SalePrice, dtype: float64

Selected features: ['OverallQual', 'GrLivArea', 'GarageCars', 'TotalBsmstSF',

Largest selected predictor correlations (training only):
TotalBsmstSF 1stFlrSF    0.827
GrLivArea    FullBath    0.616
OverallQual  GarageCars  0.592
              GrLivArea  0.584
GrLivArea    1stFlrSF    0.563
```

Рис.5 – Аналіз даних, відбір ознак для навчання

Для цільової змінної SalePrice найбільші кореляції на навчальній вибірці отримано для:

- OverallQual — 0.785;
- GrLivArea — 0.689;
- GarageCars — 0.643;
- GarageArea — 0.622;
- TotalBsmtSF — 0.590;
- 1stFlrSF — 0.583;
- FullBath — 0.549;
- TotRmsAbvGrd — 0.520;
- YearBuilt — 0.512.

Для подальшої побудови моделі було обрано 10 ознак:

- Числові: OverallQual, GrLivArea, GarageCars, TotalBsmtSF, 1stFlrSF, YearBuilt, FullBath.
- Категоріальні: Neighborhood, KitchenQual, GarageType.

Такий набір ознак охоплює різні характеристики житла: якість будинку, площу, кількість місць у гаражі, площу підвалу та першого поверху, рік побудови, кількість ванних кімнат, район та характеристики кухні й гаража.

Окремо було перевірено кореляції між вибраними числовими предикторами. Найбільшою виявилася кореляція між TotalBsmtSF та 1stFlrSF: $r = 0.827$. Це означає, що ці дві ознаки містять частково дубльовану інформацію.

Для оцінювання здатності моделі узагальнювати результати початковий набір даних було розділено на три частини:

- навчальну (Train);
- валідаційну (Validation);
- тестову (Test).

Спочатку 30% даних було виділено у відкладену вибірку, після чого її поділено навпіл між Validation і Test.

У результаті отримано:

- Train = 1022
- Validation = 219
- Test = 219

Таким чином, навчальна вибірка містить приблизно 70% початкових даних, а Validation і Test — приблизно по 15%.

Особливу увагу було приділено запобіганню data leakage. Розбиття було виконано до імпутації пропущених значень, кодування категоріальних ознак та масштабування. Це необхідно, оскільки параметри цих операцій повинні

визначатися тільки за навчальною вибіркою. Тестова вибірка при цьому залишалася невикористаною до фінального оцінювання моделі.

Оскільки лінійна регресія працює з числовими значеннями, категоріальні ознаки необхідно було перетворити у числове представлення. Для цього використано One-Hot Encoding.

Було закодовано такі категоріальні ознаки:

- Neighborhood
- KitchenQual
- GarageType

One-Hot Encoding створює окрему бінарну ознаку для кожної категорії. Такий підхід використано тому, що категорії на кшталт району або типу гаража не мають природного числового порядку.

Наприклад, присвоєння категоріям значень 1, 2, 3 могло б створити штучне припущення, що категорія 3 "більша" за категорію 1.

Для обробки категорій, які могли бути відсутніми під час навчання, використано `handle_unknown = "ignore"`. Таким чином, невідомі категорії у Validation або Test не спричиняють помилки.

Для числових ознак пропущені значення замінювалися медіаною `SimpleImputer(strategy="median")`. Для категоріальних ознак використовувалося найбільш часте значення `SimpleImputer(strategy="most_frequent")`. Усі параметри імпутації та кодування визначалися тільки на Train. Validation і Test лише перетворювалися за вже навченими правилами.

Після обробки пропущених значень числові ознаки було стандартизовано за допомогою `StandardScaler`.

Стандартизація виконується за формулою:

$$x' = (x - \mu) / \sigma$$

де:

- x — початкове значення ознаки;
- μ — середнє значення ознаки на навчальній вибірці;
- σ — стандартне відхилення ознаки.

Масштабування є особливо важливим для градієнтного спуску, оскільки різні масштаби ознак можуть суттєво впливати на напрямок і величину градієнта.

Після попередньої обробки отримано:

- `X_train = (1022, 42)`
- `X_val = (219, 42)`

- $X_{\text{test}} = (219, 42)$

Тобто після One-Hot Encoding початкові 10 відібраних ознак перетворилися на 42 числові ознаки.

Етап 2: Реалізація лінійної регресії засобами NumPy.

Після попередньої обробки було реалізовано власну модель лінійної регресії без використання готового алгоритму LinearRegression зі Scikit-learn.

Модель має вигляд:

$$\hat{y} = Xw + b$$

де X — матриця ознак, w — вектор ваг, а b — вільний член.

Для чисельної стабільності цільову змінну SalePrice також було стандартизовано на основі параметрів навчальної вибірки. Середнє значення SalePrice навчальної вибірки становило 181312.7, а стандартне відхилення — 77579.5.

У класі LinearRegressionNumpy було реалізовано основні операції:

- прогнозування;
- розрахунок функції втрат;
- обчислення градієнтів;
- оновлення ваг;
- збереження історії значень функції втрат.

Прогноз обчислюється за допомогою матричного множення:

$$X@w + b$$

Функція втрат використовує половину середньоквадратичної помилки:

$$J = 0.5 \times \text{mean}((\hat{y} - y)^2)$$

Градiente обчислюються векторизовано засобами NumPy:

$$dw = X^T(\hat{y} - y)/n$$

$$db = \text{mean}(\hat{y} - y)$$

Етап 3: Реалізація градієнтного спуску.

1. Batch Gradient Descent

У Batch GD для одного оновлення ваг використовується вся навчальна вибірка. На кожній епосі:

- обчислюється прогноз для всіх навчальних об'єктів;
- визначаються помилки;
- обчислюється загальний градієнт;
- оновлюються всі ваги;

- зберігається значення функції втрат.

У реалізації Batch GD використано 300 епох.

Перевагою такого підходу є стабільний напрямок руху градієнта, оскільки він розраховується за всіма навчальними прикладами.

Недоліком є більша обчислювальна вартість одного оновлення.

2. Stochastic Gradient Descent

У SGD оновлення параметрів виконується для одного навчального прикладу за раз.

Перед кожною епохою порядок навчальних прикладів переміщується.

Для кожного об'єкта:

- береться один рядок X ;
- виконується прогноз;
- обчислюється помилка;
- розраховується градієнт;
- одразу оновлюються ваги.

У реалізації використано 80 епох.

Головною особливістю SGD є висока частота оновлення параметрів. Через використання одного об'єкта напрямок градієнта містить більше випадкового шуму, тому крива збіжності є більш нерівною.

3. Mini-batch Gradient Descent

Mini-batch GD є проміжним варіантом між Batch GD і SGD.

На кожній епосі навчальні дані переміщуються, після чого розбиваються на невеликі групи. У роботі використано `batch_size = 32`.

Для кожного міні-батчу обчислюється градієнт та виконується оновлення параметрів. Навчання виконується протягом 150 епох.

Таким чином:

- Batch GD використовує весь датасет для одного оновлення;
- SGD використовує один об'єкт;
- Mini-batch GD використовує групу з 32 об'єктів.

Mini-batch GD дозволяє отримати компроміс між стабільністю Batch GD та швидкістю SGD.

Етап 4: Вибір швидкості навчання та аналіз збіжності.

Для кожного алгоритму було перевірено декілька значень коефіцієнта навчання: 0.0003, 0.001, 0.003, 0.01, 0.03.

Вибір проводився за валідаційною MSE, при цьому тестова вибірка не використовувалась.

Отримано такі оптимальні значення:

Метод	Найкраще α
Batch GD	0.030
SGD	0.001
Mini-batch GD	0.010

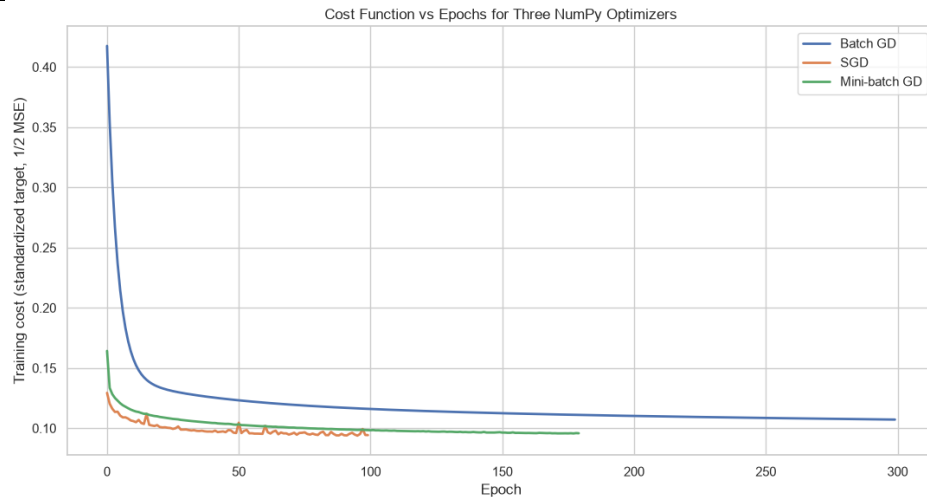


Рис.6 – Графіки навчання на різних градієнтах при оптимальних α

Етап 5: Оцінка якості моделей та коефіцієнт детермінації.

Для оцінювання якості моделей використовувався коефіцієнт детермінації R^2 . Він визначається власною функцією на NumPy:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Значення R^2 показує, яку частину варіації цільової змінної пояснює модель. У роботі оцінювання виконувалося окремо для Train, Validation та Test. Тестова вибірка використовувалась лише на фінальному етапі оцінювання.

Фінальні результати:

Метод	Train R^2	Validation R^2	Test R^2
Batch GD	0.7857	0.8742	0.7888
SGD	0.8115	0.8824	0.8094
Mini-batch GD	0.8084	0.8838	0.8097

За результатами експерименту найкращі Validation та Test показники продемонстрував Mini-batch GD. Batch GD показав найстабільнішу поведінку, однак його результат на тестовій вибірці нижчий. SGD отримав дещо вищий Train R^2 , але характеризується більшою нестабільністю процесу оптимізації. Mini-batch GD поєднав переваги двох інших методів і в цьому експерименті забезпечив найкраще загальне співвідношення якості та стабільності. Це також відповідає висновкам самого ноутбука.

Етап 6: Перевірка статистичних припущень

Після навчання найкращої моделі було виконано аналіз залишків. Залишок визначається як різниця між фактичним та прогнозованим значенням: $e_i = y_i - \hat{y}_i$. Аналіз залишків дозволяє перевірити, наскільки поведінка моделі відповідає основним припущенням лінійної регресії.

Було використано:

- графік залишків;
- Scale-Location plot;
- Q-Q plot;
- тест Шапіро–Вілка;
- аналіз мультиколінеарності за допомогою VIF.

1. Аналіз залишків

Для правильно специфікованої лінійної моделі залишки не повинні демонструвати очевидної систематичної структури.

Зокрема:

- криволінійна структура може свідчити про відсутність нелінійних компонентів;
- форма "воронки" може свідчити про неоднакову дисперсію залишків;
- окремі дуже віддалені точки можуть бути впливовими спостереженнями.

У дослідженні було виявлено, що для дорогих будинків розкид залишків збільшується. Також присутні окремі викиди. Це свідчить про порушення припущення сталої дисперсії.

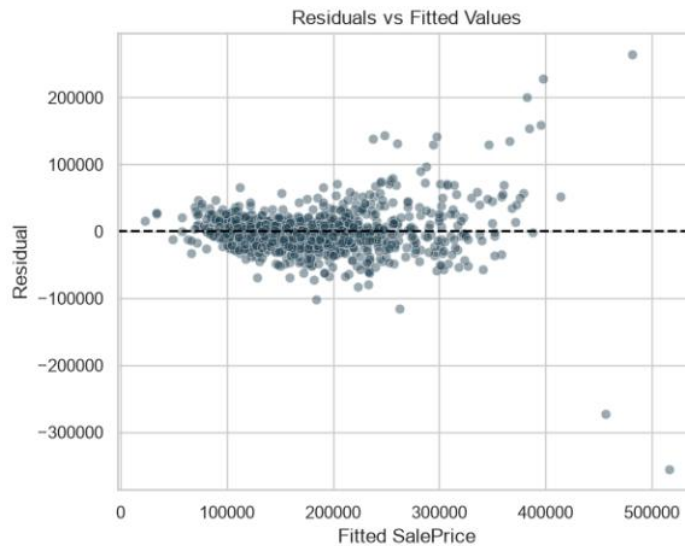


Рис.7 – Графік розкиду помилок

2. Перевірка нормальності залишків

Для перевірки нормальності було побудовано Q-Q plot та виконано тест Шапіро–Вілکا.

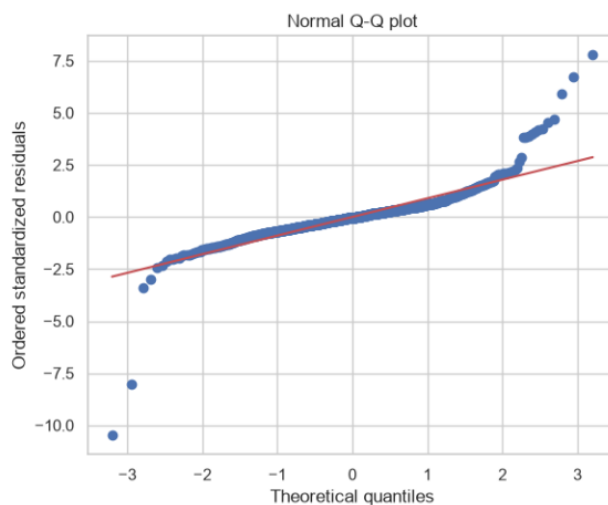


Рис.8 – Графік Q-Q plot

Отримано:

- Shapiro-Wilk statistic = 0.8027
- p-value = 1.664e-33

Оскільки $p\text{-value} < 0.001$, нульова гіпотеза про нормальність залишків відхиляється. Отже, статистично нормальність залишків не підтверджується.

3. Перевірка гомоскедастичності

Гомоскедастичність означає приблизно сталу дисперсію залишків при різних прогнозованих значеннях.

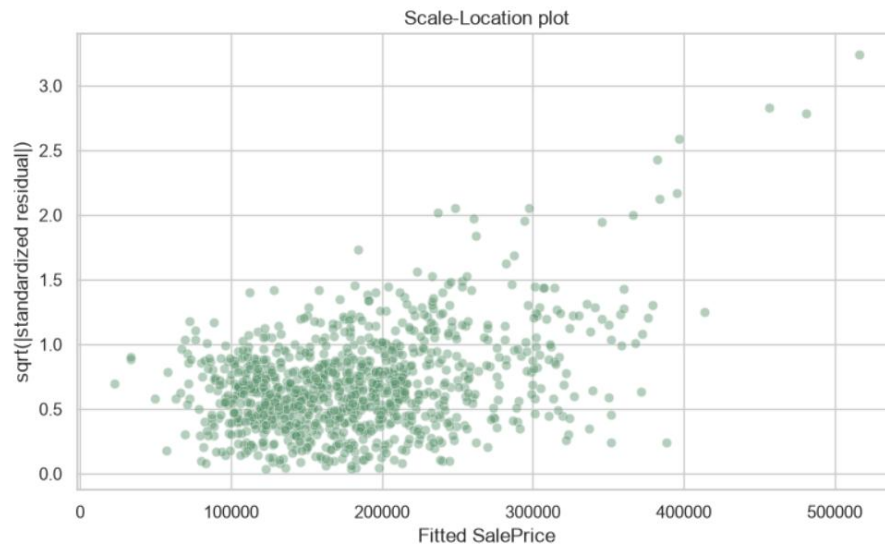


Рис.8 – Графік Scale-Location plot

За Scale-Location plot було виявлено тенденцію до збільшення розкиду залишків зі збільшенням прогнозованої вартості будинку. Отже, припущення про сталу дисперсію залишків у даній моделі порушується.

4. Перевірка мультиколінеарності

Мультиколінеарність виникає тоді, коли незалежні змінні містять значну кількість спільної інформації.

Для її оцінювання було використано VIF — Variance Inflation Factor.

Ознака	VIF
1stFlrSF	3.78
TotalBsmtSF	3.74
GrLivArea	2.72
OverallQual	2.50
FullBath	2.13
YearBuilt	2.10
GarageCars	1.82

Найбільші значення VIF отримано для 1stFlrSF та TotalBsmtSF, що узгоджується з їхньою високою кореляцією 0.827.

Водночас отримані значення VIF не свідчать про критичну мультиколінеарність, хоча вони вказують на наявність певної надлишковості інформації між ознаками. Це може ускладнювати інтерпретацію окремих коефіцієнтів моделі.

Етап 7: Додатковий експеримент із масштабуванням

Для демонстрації впливу масштабування на градієнтний спуск було проведено додатковий експеримент. Спочатку модель навчалася на немасштабованих ознаках, після чого її було порівняно з моделлю, яка використовує стандартизовані ознаки.

Без масштабування оптимальний результат було отримано при: $\alpha = 1e-07$

Валідаційна функція втрат: 0.1766

Для масштабованих ознак валідаційна функція втрат становила: 0.0591

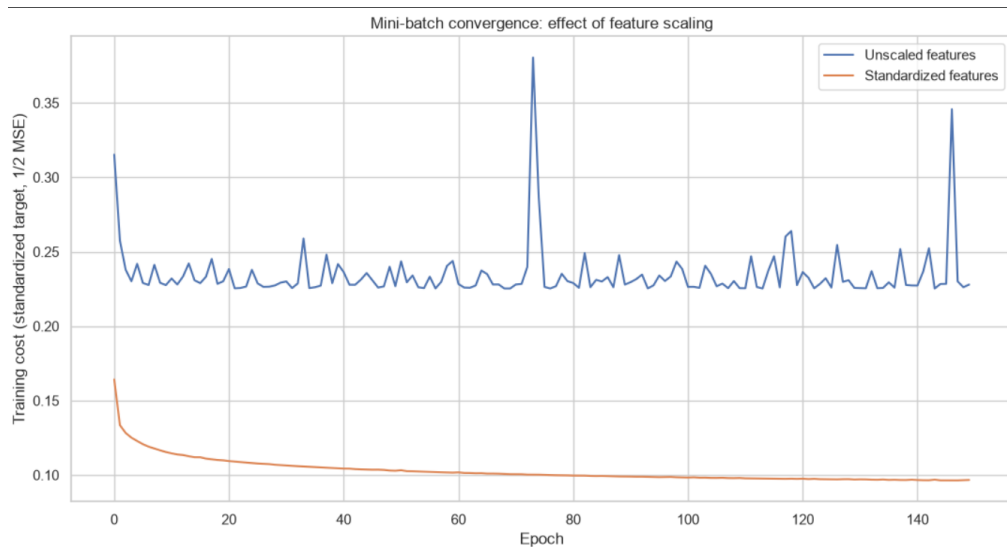


Рис.9 – Графіки впливу стандартизації на навчання моделі

Висновок

У ході виконання лабораторної роботи було досліджено процес побудови моделі лінійної регресії для прогнозування вартості житлових будинків на основі набору даних Ames Housing. Було проведено первинний аналіз структури даних, визначено числові та категоріальні ознаки, досліджено пропущені значення та основні статистичні характеристики змінних.

Було виконано відбір ознак на основі їхнього зв'язку з цільовою змінною SalePrice. Для побудови моделі обрано 7 числових та 3 категоріальні ознаки. Дані було розділено на навчальну, валідаційну та тестову вибірки. Попередня обробка виконувалася без витоку інформації з Validation і Test: параметри імпутації, кодування та масштабування визначалися лише за навчальною вибіркою.

Для роботи з категоріальними ознаками застосовано One-Hot Encoding, а числові ознаки було стандартизовано. Після кодування початкові 10 відібраних характеристик перетворилися на 42 числові ознаки, придатні для використання в математичній моделі.

Було реалізовано лінійну регресію засобами NumPy та досліджено три методи її оптимізації: Batch Gradient Descent, Stochastic Gradient Descent та Mini-batch Gradient Descent. Експеримент показав, що Batch GD забезпечує найбільш стабільний процес збіжності, SGD характеризується найбільшими коливаннями функції втрат, а Mini-batch GD забезпечує компроміс між стабільністю та швидкістю навчання.

За результатами оцінювання найкращі показники на валідаційній та тестовій вибірках продемонстрував Mini-batch GD: Validation $R^2 = 0.8838$ та Test $R^2 = 0.8097$. Це дозволяє вважати його найбільш ефективним із трьох досліджених методів для використаного набору даних та параметрів експерименту.

Також було досліджено вплив масштабування ознак на процес оптимізації. Порівняння показало, що стандартизація дозволяє отримати суттєво менше значення функції втрат на валідаційній вибірці, що підтверджує важливість приведення ознак до порівнянного масштабу під час використання градієнтного спуску.

У результаті статистичного аналізу було встановлено, що залишки моделі не повністю відповідають теоретичним припущенням класичної лінійної регресії. Тест Шапіро–Вілка показав статистично значуще відхилення від нормального розподілу залишків ($p\text{-value} = 1.664\text{e-}33$). Аналіз графіків залишків також виявив збільшення їхнього розкиду для дорожчих будинків та наявність окремих викидів. Перевірка VIF показала наявність певної кореляції між окремими ознаками, зокрема між TotalBsmtSF та 1stFlrSF, однак критичної мультиколінеарності не виявлено.

Таким чином, у лабораторній роботі було реалізовано повний цикл побудови та дослідження моделі лінійної регресії: від аналізу та попередньої обробки даних до реалізації різних методів градієнтного спуску, оцінювання якості моделі та перевірки статистичних припущень. Отримані результати показали, що правильний відбір ознак, відсутність витоку даних, кодування категоріальних змінних та масштабування суттєво впливають на якість і стабільність навчання моделі.

Код програми

Код до даної лабораторної роботи розміщений у репозиторії https://github.com/MrCatman137/Machine_learning у директорії lab_1.